Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №3
з дисципліни «Методи та технології паралельного програмування»

Тема: «Засоби взаємодії процесів в паралельних програмах з розподіленою пам'яттю»

Варіант №4

Виконав:
студент 1 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
асистент, Лесик Валентин Олександрович

Дата здачі: 09.04.2026

КИЇВ – 2026

Мета роботи. Ознайомитись з моделлю паралельного програмування з розподіленою пам'яттю, вивчити принципи обміну повідомленнями між процесами (Message Passing) та порівняти ефективність з моделлю спільної пам'яті, на прикладі проблем взаємного блокування (Deadlock) та стану гонитви (Race Condition).

Завдання. Реалізувати послідовну та паралельну мультипоточну обробку незалежних задач, за допомогою використання відповідних патернів паралельного програмування. Необхідно обрати відповідний фреймворк, описати його можливості та обмеження, пояснити причину вибору фреймворку відповідно описаним обмеженням. Мова програмування обирається на розгляд студента. Результати роботи повинні містити виконання логіки програми без застосування паралелізації та із застосуванням паралелізації за різних гіперпараметрів: кількості потоків, процесів, worker-ів; для кожного рішення треба знайти час виконання та зробити висновки, що впливає на прискорення та які треба підбирати гіперпараметри. У відповідях на контрольні запитання слід вказувати на особливості та відмінності обраного студентом фреймворку, так як в залежності від обраної мови буде різнитися реалізація, підходи та можливості застосування.

Задача 1: Deadlock та Race Condition. ***Переказ коштів між банківськими рахунками.***

Створити > 100 рахунків із випадковою кількістю коштів; підрахувати кількість всіх коштів до початку переказів; запустити > 1000 потоків, що переказують кошти між двома випадковими акаунтами на протязі деякого часу (врахувати, що сума на рахунку не повинна бути від'ємною); підрахувати загальну кількість коштів після завершення всіх транзакцій. Реалізувати два варіанти програми – із рішенням проблеми та без рішення. Продемонструвати де в програмі з'являється проблема Deadlock, Race Condition.

Задача 2: Передача даних між процесами різного роду.

Вибрати два різних фреймворки паралельного програмування на різних мовах програмування, створити основний процес та допоміжний за допомогою різних методів. Основний процес повинен генерувати випадкове число, передавати його в інший процес, де відбувається логування даного числа і отримувати його назад. Обрати три методи даних (Shared Memory, Message Passing та ін.) між двома процесами, їх обрати самостійно, дослідивши існуючі методи. Розробити програму у одному середовищі та в двох, проаналізувати проблемні області та потенційні втрати часу через передачу даних, зробити висновки.

Задача 3: Дослідження.

Практично продемонструвати класичні проблеми паралельного доступу до даних та їх рішення. Дослідити методи передачі даних між процесами із розподіленою та спільною пам'яттю, продемонструвати переваги, недоліки, часові витрати, недоліки імплементації та застосування.

Вимоги до звіту. Повинна бути титульна сторінка, тема, мета, завдання, короткий опис обраного підходу до рішення задачі, опис критичних моментів рішення, код проекту (лістинг), обґрунтування обрання фреймворку, опис відмінностей та переваг, скріншоти із демонстрацією виконання програми, скріншоти із демонстрацією прискорення та різними значеннями параметрів, відповіді на контрольні запитання, висновки.

Хід роботи.

Задача 1: Deadlock та Race Condition. ***Переказ коштів між банківськими рахунками.***

Опис підходу до вирішення.

Для демонстрації класичних проблем багатопотоковості було розроблено симулятор банківських транзакцій на мові Rust. Програма ініціалізує 150 рахунків із псевдовипадковими початковими балансами та надійно обчислює загальний капітал системи до початку операцій. Паралельна обробка переказів виконується пулом із 5000 незалежних потоків. Оскільки ресурси є спільними, доступ до балансу кожного рахунку захищено комбінацією атомарного лічильника посилань та м'ютекса. Реалізовано три логічні гілки виконання. Для симуляції стану гонитви (Race Condition) транзакція розбивається на неатомарні кроки читання та запису. Для симуляції взаємного блокування (Deadlock) застосовано патерн циклічного очікування, де потоки конкурують за два рахунки у випадковому порядку. Вирішення обох проблем реалізовано в третьому режимі за допомогою утримання блокування на весь час транзакції (вирішує стан гонитви) та застосування архітектурного підходу глобального впорядкування блокувань (вирішує взаємне блокування).

2026-09-04.log

```
[2026-04-09 03:22 INFO] [Race Condition] Initial Funds: 88475. Final Funds: 88465. Difference: -10
[2026-04-09 03:22 WARN] Deadlock successfully detected! Execution halted to prevent permanent freeze.
[2026-04-09 03:22 INFO] [Mutex (Fixed)] Initial Funds: 88475. Final Funds: 88475. Difference: 0
```

Критичні моменти рішення.

Важливим інженерним викликом стала сама можливість реалізації класичних помилок синхронізації у безпечній підмножині (Safe Rust), оскільки система типів та строгі правила володіння пам'яттю за замовчуванням захищають розробника від гонок даних (data races). Для штучного створення стану гонитви довелося явно розірвати критичну секцію: зчитати баланс, примусово скинути блокування (`drop(balance)`), віддати час процесора планувальнику через `thread::yield_now()` і лише потім перезаписати стан. Іншою проблемою виявилася недетермінованість взаємного блокування. При високій швидкості потоків дедлок міг випадковим чином не виникати. Для його стовідсоткового відтворення між захопленням першого та другого м'ютексів було додано затримку `thread::sleep(Duration::from_millis(1))`. Щоб уникнути вічного зависання всієї програми під час бенчмаркінгу, проблемний сценарій було обгорнуто в ізольований потік, за яким головний процес стежив через систему обміну повідомленнями `mpsc` із тайм-аутом `recv_timeout` у дві секунди.

Аналіз результатів та порівняння патернів.

Згідно з отриманими метриками бенчмаркінгу, режим обробки Race Condition відпрацював найшвидше – за 0.0990 секунди. Проте цей результат є хибним: через перезапис проміжних значень паралельними потоками фінальна перевірка цілісності даних виявила незворотну втрату 10 одиниць коштів. Режим Deadlock очікувано призвів до повного паралічу системи через циклічне очікування м'ютексів і був штатно перерваний захисним тайм-аутом через 2.0004 секунди. Виправлений варіант програми успішно завершив усі 5000 транзакцій за 0.1082 секунди. Різниця між початковим та фінальним капіталом склала рівно 0. Правило глобального впорядкування (потоки завжди блокують рахунок з меншим ідентифікатором перед рахунком з більшим) математично унеможливило дедлок. Ці результати наочно доводять, що коректне використання м'ютексів із правильним архітектурним плануванням додає мізерні накладні витрати (менше 0.01 секунди порівняно з «небезпечним» кодом), але гарантує стовідсоткову надійність та атомарність операцій над спільними даними.

```
=== REPORT FOR BANK TRANSACTIONS ===
Race Condition: 0.0990 seconds.
Deadlock: 2.0004 seconds.
Mutex (Fixed): 0.1082 seconds.
```

Задача 2: Передача даних між процесами різного роду. та **Задача 3: Дослідження.**

Опис підходу до вирішення.

Для дослідження методів міжпроцесної взаємодії (IPC) було розроблено гетерогенну двокомпонентну систему. Основний процес (хост) написано мовою системного програмування Rust, що забезпечує низькорівневий контроль над пам'яттю та системними викликами операційної системи. Допоміжний процес (воркер) реалізовано на платформі .NET мовою C#, що дозволило продемонструвати інтеграцію принципово різних середовищ виконання. Основний процес відповідає за прекомпіляцію воркера, генерацію псевдовипадкового числа з рухомою комою подвійної точності (64-бітне) та замір загального часу виконання. Передача даних була реалізована трьома незалежними методами. Перший метод – Message Passing через стандартні потоки вводу/виводу (Named Pipes), де процеси спілкуються за допомогою рядкових повідомлень. Другий метод – локальні мережеві сокети (TCP), де Rust відкриває сервер на випадковому вільному порту, а C# підключається як клієнт для двостороннього обміну сирими байтами. Третій метод – розділювана пам'ять (Shared Memory), реалізована через відображення тимчасового файлу в оперативну пам'ять (Memory-Mapped Files), що дозволяє обом процесам читати та записувати дані за однією фізичною адресою без посередництва ядра операційної системи під час самої передачі.

```
ipc.rs    Program.cs    2026-09-04.log ×
1  [2026-04-09 23:15 INFO] Building C# Worker project in Release mode...
2  [2026-04-09 23:15 INFO] [SHM] Rust sent: 0.7275, Received back: -0.7275
3  [2026-04-09 23:15 INFO] Building C# Worker project in Release mode...
4  [2026-04-09 23:15 INFO] [Pipes] Rust sent: 0.7275, Received back: 0.727535365860596
5  [2026-04-09 23:15 INFO] Building C# Worker project in Release mode...
6  [2026-04-09 23:15 INFO] [TCP] Rust sent: 0.7275, Received back: 0.7275
```

Критичні моменти рішення.

Важливим інженерним викликом стала проблема бінарної сумісності та серіалізації даних між середовищами Rust та .NET. Для TCP та Shared Memory замість повільної конвертації у текст було обрано пряму передачу байтів (to_ne_bytes). Це вимагало суворого контролю над порядком байтів (endianness) та розміром типів даних (8 байт для IEEE 754 f64). Іншим критичним моментом стала синхронізація у методі розділюваної пам'яті. Оскільки Shared Memory сама по собі не надає механізмів сповіщення про зміну даних, у процесі Rust було реалізовано патерн активного опитування (active polling) із мікро-затримкою в одну

мілісекунду. Rust нескінченно читає спільну ділянку пам'яті, чекаючи, поки C# процес не змінить знак переданого числа на від'ємний, що слугує прапорцем успішної обробки. Також довелося відмовитись від відносних шляхів та використовувати жорстко задані константи для конвеєра збірки MSBuild, щоб гарантувати коректний запуск допоміжного процесу незалежно від поточної робочої директорії терміналу.

Аналіз результатів та порівняння методів.

Метрики продуктивності показали несподівано близькі результати для всіх трьох методів: використання іменованих каналів (Pipes) зайняло 0.3977 секунди, TCP-сокети відпрацювали за 0.4149 секунди, а розділювана пам'ять – за 0.4173 секунди. Така мінімальна різниця пояснюється тим, що загальний час бенчмарку на 99% складається з накладних витрат на запуск віртуальної машини .NET (CLR) через системний виклик створення нового процесу дочірньою командою. Теоретично, метод спільної пам'яті є найшвидшим, оскільки обмін даними відбувається зі швидкістю роботи оперативної пам'яті без копіювання буферів у простір ядра ОС. Проте на практиці активне опитування та ініціалізація файлового мапінгу додали незначну затримку, зробивши цей метод у нашому тесті формально «найповільнішим». Передача через стандартні канали виявилася найбільш зручною у розробці та найшвидшою у цьому конкретному сценарії завдяки вбудованій системній оптимізації пайпів. Локальні TCP-сокети показали середній результат і найбільшу алгоритмічну складність, але вони є єдиним з досліджених методів, який дозволяє прозоро масштабувати архітектуру на декілька фізичних серверів у розподіленій мережі без зміни логіки передачі даних.

```
=== REPORT FOR INTER-PROCESS COMMUNICATION ===
IPC: Shared Memory: 0.4173 seconds.
IPC: Named Pipes: 0.3977 seconds.
IPC: TCP Sockets: 0.4149 seconds.
```

Висновок. Під час виконання лабораторної роботи було практично досліджено фундаментальні проблеми паралельного програмування та методи їх вирішення. На прикладі симуляції банківської системи мовою Rust вдалося наочно продемонструвати, як відсутність належної синхронізації призводить до стану гонитви та незворотного спотворення спільних даних. Водночас штучне створення взаємного блокування показало критичну небезпеку циклічного очікування ресурсів. Застосування глобального впорядкування м'ютексів довело свою абсолютну

ефективність, повністю усунувши обидві проблеми та забезпечивши стовідсоткову транзакційну цілісність із мінімальними накладними витратами часу.

Дослідження міжпроцесної взаємодії між середовищами Rust та .NET дозволило порівняти ефективність різних методів передачі даних. Хоча теоретично розділювана пам'ять забезпечує найшвидший апаратний обмін інформацією, практичний бенчмаркінг показав, що накладні витрати на запуск віртуальної машини CLR дочірнього процесу та необхідність ручної синхронізації через активне опитування нівелюють цю перевагу для швидких одиничних транзакцій. У подібних локальних сценаріях стандартні іменовані канали (Pipes) виявилися найбільш збалансованим та швидким рішенням завдяки вбудованій системній оптимізації, тоді як мережеві сокети довели свою алгоритмічну доцільність переважно для подальшого масштабування у розподілених системах. Загалом, результати роботи підтверджують, що успішна розробка високопродуктивних паралельних систем вимагає ретельного архітектурного планування стратегій доступу до ресурсів та обрання оптимальних транспортних каналів зв'язку відповідно до специфіки задачі.

Контрольні запитання.

1. **У чому різниця між моделлю спільної пам'яті та розподіленої пам'яті в паралельному програмуванні?**
   Модель спільної пам'яті передбачає, що всі потоки виконання мають безпосередній доступ до єдиного фізичного або віртуального адресного простору, де зміни, зроблені одним потоком, миттєво стають доступними для зчитування іншими. У моделі розподіленої пам'яті кожен окремий процес володіє власним ізольованим адресним простором, а будь-який обмін інформацією відбувається виключно через явне надсилання та отримання структурованих повідомлень по комунікаційних каналах або комп'ютерній мережі.
2. **Які основні переваги та недоліки підходів shared memory і message passing з точки зору масштабованості та складності розробки?**
   Підхід спільної пам'яті забезпечує надзвичайно високу швидкість взаємодії та мінімальні затримки, проте його вкрай складно масштабувати за межі одного фізичного комп'ютера, а сам процес розробки вимагає ретельної та складної синхронізації для уникнення станів гонки та взаємних блокувань. Обмін повідомленнями, навпаки, легко та природно масштабується на великі кластери та розподілені хмарні системи, роблячи архітектуру більш передбачуваною, але цей підхід вимагає додаткових обчислювальних витрат

на серіалізацію даних та передачу їх через мережеві або міжпроцесні транспортні канали.

3. **Які операції над спільними даними є типово неатомарними і часто призводять до race condition?**

   До стану гонки найчастіше призводять будь-які операції циклу читання-модифікації-запису. Наприклад, класичне зчитування поточного фінансового балансу з оперативної пам'яті у регістр процесора, подальше додавання до нього певної суми та зворотний запис зміненого значення у пам'ять не є атомарним процесом за своєю природою. Якщо планувальник операційної системи раптово перемикає контекст потоку між цими кроками, інший потік може прочитати старе значення, змінити його і записати першим, після чого початковий потік неминуче перезапише ці дані своїм результатом, що гарантовано призведе до втрати однієї з проведених транзакцій.

4. **Які механізми синхронізації у вибраному вами фреймворку використовуються для усунення race condition та deadlock?**

   У мові Rust для надійного захисту спільних структур даних використовується системний примітив синхронізації `Mutex`, який апаратно гарантує ексклюзивний доступ до ресурсу лише одному потоку одночасно. Для повного усунення стану гонки м'ютекс утримується у заблокованому стані нерозривно протягом усієї логічної транзакції переказу коштів. Для уникнення взаємного блокування застосовується строгий архітектурний підхід глобального впорядкування доступу, за якого всі потоки без винятку завжди блокують наявні м'ютекси у фіксованому порядку від найменшого ідентифікатора рахунку до найбільшого.

5. **Які практичні підходи до уникнення deadlock існують?**

   Найбільш поширені практичні підходи включають імплементацію глобального впорядкування ресурсів, застосування сучасних неблокуючих алгоритмів із вбудованими механізмами тайм-аутів для відмови від захоплення ресурсу при тривалому очікуванні, а також використання спеціалізованих алгоритмів виявлення та примусового розв'язання дедлоків у режимі реального часу. Надійним альтернативним підходом є повна або часткова відмова від використання спільних м'ютексів на користь асинхронної моделі акторів або транзакційної пам'яті програмного забезпечення.

6. **Які три методи обміну даними між процесами ви реалізували/дослідили? Які їх особливості, обмеження, переваги? Які підходи дали найменші затримки передачі даних і чому?**

У рамках другої задачі передбачено дослідження трьох класичних методів обміну інформацією. Метод розділюваної пам’яті забезпечує найменші можливі затримки передачі даних завдяки прямому апаратному доступу до сторінок оперативної пам’яті без копіювання, але він покладає всю відповідальність за ручну синхронізацію на розробника. Метод каналів надає зручний та безпечний потоковий інтерфейс для тісної взаємодії процесів у межах однієї операційної системи, беручи на себе всі питання буферизації даних. Метод локальних мережевих сокетів демонструє найбільші накладні витрати часу через необхідність проходження даних через весь системний мережевий стек, проте він єдиний надає прозору можливість у майбутньому розділити взаємодіючі процеси між різними фізичними машинами.

Лістинг.

bank.rs

```rust
use crate::errors::Error;
use crate::task::benchmark::{BenchmarkKind, Benchmarkable};
use crate::task::executable::{Executable, RunMode};
use crate::task::lifecycle::Manageable;
use crate::task::measure::Measurable;
use crate::task::report::Reportable;
use crate::tasks::TaskLogicError;
use std::sync::{Arc, Mutex};
use std::thread;
use std::time::Duration;

#[derive(Debug, Clone)]
pub struct Account {
    pub id: usize,
    pub balance: Arc<Mutex<i64>>,
}

#[derive(Debug)]
pub struct Bank {
    accounts: Vec<Account>,
    accounts_amount: usize,
    initial_total: i64,
}

const THREAD_AMOUNT: usize = 5000;

impl Bank {
    pub fn new(accounts_amount: usize) -> Self {
        Self {
            accounts: vec![],
            accounts_amount,
            initial_total: 0,
        }
    }

    // Calculates the total money across all accounts to verify data integrity
    fn calculate_total_balance(&self) -> Result<i64, Error> {
        let mut total = 0;
        for account in &self.accounts {
            total += *account
            .balance
            .lock()
            .map_err(|_| TaskLogicError::MutexPoisoned)?;
        }

        Ok(total)
    }

    // Generates a deterministic pseudo-random index for transferring
    fn get_account_pair(&self, iteration: usize) -> (Account, Account) {
        let first_id = (iteration * 17) % self.accounts_amount;
        let second_id = (iteration * 31) % self.accounts_amount;

        let second_id = if first_id == second_id {
            (second_id + 1) % self.accounts_amount
        } else {
            second_id
        };
```

```rust

        let first_account = self.accounts[first_id].clone();
        let second_account = self.accounts[second_id].clone();

        (first_account, second_account)
    }
}

impl Reportable for Bank {
    fn name(&self) -> &'static str {
        "Bank Transactions"
    }
}

impl Measurable for Bank {}

impl Benchmarkable for Bank {
    fn benchmarks(&self) -> Vec<BenchmarkKind> {
        vec![
        BenchmarkKind::RaceCondition,
        BenchmarkKind::Deadlock,
        BenchmarkKind::MutexFixed,
        ]
    }
}

impl Manageable for Bank {
    fn setup(&mut self) -> Result<(), Error> {
        self.accounts.clear();
        self.initial_total = 0;

        for id in 0..self.accounts_amount {
            // Assigning a pseudo-random initial balance between 100 and 1100
            let balance = 100 + (id * 37 % 1000) as i64;
            self.initial_total += balance;

            self.accounts.push(Account {
                id,
                balance: Arc::new(Mutex::new(balance)),
            });
        }

        Ok(())
    }

    fn teardown(&mut self) -> Result<(), Error> {
        self.accounts.clear();
        Ok(())
    }
}

impl Executable for Bank {
    fn supported_modes(&self) -> Vec<RunMode> {
        vec![
        RunMode::RaceCondition,
        RunMode::Deadlock,
        RunMode::MutexFixed,
        ]
    }

    fn run(&self, kind: BenchmarkKind) -> Result<(), Error> {
        match kind {
            BenchmarkKind::RaceCondition => self.run_race_condition()?,
```

```
            BenchmarkKind::Deadlock => self.run_deadlock()?,
            BenchmarkKind::MutexFixed => self.run_mutex_fixed()?,
            _ => unreachable!(),
        };

        // Skip total balance validation for Deadlock because background threads
        // will hold the locks forever, causing the main thread to freeze here.
        if !matches!(kind, BenchmarkKind::Deadlock) {
            let final_total = self.calculate_total_balance()?;
            log::info!(
            "[{}] Initial Funds: {}. Final Funds: {}. Difference: {}",
            kind,
            self.initial_total,
            final_total,
            final_total - self.initial_total
            );
        }

        Ok(())
    }
}

impl Bank {
    fn run_race_condition(&self) -> Result<(), Error> {
        let mut handles = vec![];

        for i in 0..THREAD_AMOUNT {
            let (from, to) = self.get_account_pair(i);
            let transaction_diff = 10;

            handles.push(thread::spawn(move || -> Result<(), TaskLogicError> {
                // Simulating a data race by dropping the lock between read and write
                let balance_from = from
                .balance
                .lock()
                .map_err(|_| TaskLogicError::MutexPoisoned)?;
                if *balance_from >= transaction_diff {
                    let old_from = *balance_from;
                    drop(balance_from);

                    let balance_to = to
                    .balance
                    .lock()
                    .map_err(|_| TaskLogicError::MutexPoisoned)?;
                    let old_to = *balance_to;
                    drop(balance_to);

                    thread::yield_now();

                    *from
                    .balance
                    .lock()
                    .map_err(|_| TaskLogicError::MutexPoisoned)? =
                    old_from - transaction_diff;
                    *to.balance
                    .lock()
                    .map_err(|_| TaskLogicError::MutexPoisoned)? =
                    old_to + transaction_diff;
                }

                Ok(())
            }));
        }
```

```rust

        // If there are errors, return them
        for handle in handles {
            let thread_logic_result = handle
            .join()
            .map_err(|_| TaskLogicError::JoinThreadFailed)?;
            thread_logic_result?;
        }

        Ok(())
    }

    fn run_deadlock(&self) -> Result<(), Error> {
        let (tx, rx) = std::sync::mpsc::channel();

        let accounts = self.accounts.clone();
        let accounts_amount = self.accounts_amount;

        // Executing the deadlock scenario inside an isolated thread to prevent permanent freeze
        thread::spawn(move || -> Result<(), TaskLogicError> {
            let mut handles = vec![];

            for i in 0..THREAD_AMOUNT {
                let from_id = (i * 17) % accounts_amount;
                let mut to_id = (i * 31) % accounts_amount;
                if from_id == to_id {
                    to_id = (to_id + 1) % accounts_amount;
                }

                let account_from = accounts[from_id].clone();
                let account_to = accounts[to_id].clone();

                handles.push(thread::spawn(move || -> Result<(), TaskLogicError> {
                    let _lock1 = account_from
                    .balance
                    .lock()
                    .map_err(|_| TaskLogicError::MutexPoisoned)?;

                    // Forcing a context switch to heavily increase deadlock probability
                    thread::sleep(Duration::from_millis(1));

                    let _lock2 = account_to
                    .balance
                    .lock()
                    .map_err(|_| TaskLogicError::MutexPoisoned)?;

                    Ok(())
                }));
            }

            for handle in handles {
                let thread_logic_result = handle
                .join()
                .map_err(|_| TaskLogicError::JoinThreadFailed)?;
                thread_logic_result?;
            }

            let _ = tx.send(());

            Ok(())
        });

        // If the threads do not finish within 2 seconds, we successfully demonstrated a deadlock
```

```rust
        if rx.recv_timeout(Duration::from_secs(2)).is_err() {
            log::warn!(
            "Deadlock successfully detected! Execution halted to prevent permanent freeze."
            );
        }

        Ok(())
    }

    fn run_mutex_fixed(&self) -> Result<(), Error> {
        let mut handles = vec![];

        for i in 0..THREAD_AMOUNT {
            let (from, to) = self.get_account_pair(i);
            let amount = 10;

            handles.push(thread::spawn(move || -> Result<(), TaskLogicError> {
                // Global lock ordering avoids circular waits.
                // We lock the account with the smaller ID first, then the larger ID.
                let (from, to) = if from.id < to.id {
                    (from, to)
                } else {
                    (to, from)
                };

                let mut guard_from = from
                .balance
                .lock()
                .map_err(|_| TaskLogicError::MutexPoisoned)?;
                let mut guard_to = to
                .balance
                .lock()
                .map_err(|_| TaskLogicError::MutexPoisoned)?;

                if *guard_from >= amount {
                    *guard_from -= amount;
                    *guard_to += amount;
                }

                Ok(())
            }));
        }

        for handle in handles {
            let thread_logic_result = handle
            .join()
            .map_err(|_| TaskLogicError::JoinThreadFailed)?;
            thread_logic_result?;
        }

        Ok(())
    }
}
```

ipc.rs

```rust
use crate::errors::{Error, SystemError};
use crate::task::benchmark::{BenchmarkKind, Benchmarkable};
use crate::task::executable::{Executable, RunMode};
use crate::task::lifecycle::Manageable;
use crate::task::measure::Measurable;
use crate::task::report::Reportable;
```

```rust
use crate::tasks::TaskLogicError;
use memmap2::MmapMut;
use std::fs::OpenOptions;
use std::io::{BufRead, BufReader, Read, Write};
use std::net::TcpListener;
use std::process::{Command, Stdio};
use std::time::Duration;

#[derive(Debug)]
pub struct InterProcessCommunication {
    random_number: f64,
}

const SHARED_MEMORY_FILE_NAME: &str = "shm.dat";
const PATH_TO_CSHARP_PROJECT: &str = "./Lab3/CSharpWorker/CSharpWorker.csproj";

impl Default for InterProcessCommunication {
    fn default() -> Self {
        let random_number = rand::random::<f64>();

        Self { random_number }
    }
}

impl Reportable for InterProcessCommunication {
    fn name(&self) -> &'static str {
        "Inter-Process Communication"
    }
}

impl Measurable for InterProcessCommunication {}

impl Benchmarkable for InterProcessCommunication {
    fn benchmarks(&self) -> Vec<BenchmarkKind> {
        vec![
        BenchmarkKind::IpcSharedMemory,
        BenchmarkKind::IpcPipes,
        BenchmarkKind::IpcTcp,
        ]
    }
}

impl Manageable for InterProcessCommunication {
    fn setup(&mut self) -> Result<(), Error> {
        log::info!("Building C# Worker project in Release mode...");

        let status = Command::new("dotnet")
        .args(["build", "-c", "Release", PATH_TO_CSHARP_PROJECT])
        .status()
        .map_err(SystemError::ChildProcess)?;

        if !status.success() {
            return Err(TaskLogicError::ChildProcessStatus(status.to_string()))?;
        }

        Ok(())
    }

    fn teardown(&mut self) -> Result<(), Error> {
        // Cleaning up the temporary shared memory file
        let _ = std::fs::remove_file(SHARED_MEMORY_FILE_NAME);
        Ok(())
    }
```

```rust
}

impl Executable for InterProcessCommunication {
    fn supported_modes(&self) -> Vec<RunMode> {
        vec![RunMode::IpcSharedMemory, RunMode::IpcPipes, RunMode::IpcTcp]
    }

    fn run(&self, kind: BenchmarkKind) -> Result<(), Error> {
        match kind {
            BenchmarkKind::IpcSharedMemory => self.run_shared_memory()?,
            BenchmarkKind::IpcPipes => self.run_pipes()?,
            BenchmarkKind::IpcTcp => self.run_tcp()?,
            _ => unreachable!(),
        };

        Ok(())
    }
}

impl InterProcessCommunication {
    fn run_pipes(&self) -> Result<(), Error> {
        let mut child = Command::new("dotnet")
        .args([
        "run",
        "--project",
        PATH_TO_CSHARP_PROJECT,
        "-c",
        "Release",
        "--no-build",
        "--",
        "pipe",
        ])
        .stdin(Stdio::piped())
        .stdout(Stdio::piped())
        .spawn()
        .map_err(SystemError::ChildProcess)?;

        let mut stdin = child.stdin.take().ok_or(SystemError::StdIn)?;
        let stdout = child.stdout.take().ok_or(SystemError::StdOut)?;

        // Sending the generated number to the C# process
        writeln!(stdin, "{}", self.random_number).map_err(SystemError::Pipe)?;
        drop(stdin);

        // Reading the response back from the C# process
        let mut reader = BufReader::new(stdout);
        let mut response = String::new();
        reader
        .read_line(&mut response)
        .map_err(TaskLogicError::ChildAnswer)?;

        log::info!(
        "[Pipes] Rust sent: {:.4}, Received back: {}",
        self.random_number,
        response.trim()
        );

        let _ = child.wait();
        Ok(())
    }

    fn run_tcp(&self) -> Result<(), Error> {
        let listener = TcpListener::bind("127.0.0.1:0").map_err(SystemError::TcpBind)?;
```

```
        let port = listener
        .local_addr()
        .map_err(SystemError::TcpLocalAddr)?
        .port();

        let mut child = Command::new("dotnet")
        .args([
        "run",
        "--project",
        PATH_TO_CSHARP_PROJECT,
        "-c",
        "Release",
        "--no-build",
        "--",
        "tcp",
        &port.to_string(),
        ])
        .spawn()
        .map_err(SystemError::ChildProcess)?;

        // Accepting the connection from the C# worker
        let (mut stream, _) = listener.accept().map_err(SystemError::TcpConnection)?;

        let bytes_to_send = self.random_number.to_ne_bytes();
        stream
        .write_all(&bytes_to_send)
        .map_err(SystemError::BufferWrite)?;

        let mut buffer = [0u8; 8];
        stream
        .read_exact(&mut buffer)
        .map_err(SystemError::BufferRead)?;

        let received = f64::from_ne_bytes(buffer);
        log::info!(
        "[TCP] Rust sent: {:.4}, Received back: {:.4}",
        self.random_number,
        received
        );

        let _ = child.wait();
        Ok(())
    }

    fn run_shared_memory(&self) -> Result<(), Error> {
        let file = OpenOptions::new()
        .read(true)
        .write(true)
        .create(true)
        .truncate(true)
        .open(SHARED_MEMORY_FILE_NAME)
        .map_err(SystemError::SharedMemoryFile)?;

        file.set_len(8)
        .map_err(SystemError::SharedMemoryFileLength)?;

        let mut memory_mapped_buffer = unsafe {
            MmapMut::map_mut(&file).map_err(SystemError::SharedMemoryFileObject)?
        };

        // Writing the random f64 number directly into the shared memory region
        memory_mapped_buffer[0..8].copy_from_slice(&self.random_number.to_ne_bytes());

```

```rust
        let mut child = Command::new("dotnet")
        .args([
        "run",
        "--project",
        PATH_TO_CSHARP_PROJECT,
        "-c",
        "Release",
        "--no-build",
        "--",
        "shm",
        SHARED_MEMORY_FILE_NAME,
        ])
        .spawn()
        .map_err(SystemError::ChildProcess)?;

        // Active polling to detect when the C# worker negates the number
        loop {
            let current_value = f64::from_ne_bytes(
            memory_mapped_buffer[0..8]
            .try_into()
            .map_err(TaskLogicError::FloatFromMemory)?,
            );
            if current_value < 0.0 {
                log::info!(
                "[SHM] Rust sent: {:.4}, Received back: {:.4}",
                self.random_number,
                current_value
                );
                break;
            }
            std::thread::sleep(Duration::from_millis(1));
        }

        let _ = child.wait();
        Ok(())
    }
}
```

Program.cs

```csharp
using System.IO.MemoryMappedFiles;
using System.Net.Sockets;

namespace CSharpWorker;

class Program
{
    private static void Main(string[] args)
    {
        if (args.Length == 0) return;
        var mode = args[0];

        try
        {
            switch (mode)
            {
                case "pipe":
                RunPipeMode();
                break;
                case "tcp" when args.Length > 1:
                RunTcpMode(int.Parse(args[1]));
                break;
```

```
                case "shm" when args.Length > 1:
                RunSharedMemoryMode(args[1]);
                break;
            }
        }
        catch (Exception ex)
        {
            Console.Error.WriteLine($"[C# Error] {ex.Message}");
        }
    }

    private static void RunPipeMode()
    {
        // Reading the number from standard input line by line
        var input = Console.ReadLine();
        if (!double.TryParse(input, out var number)) return;
        Console.Error.WriteLine($"[C# Worker] Received via Pipes: {number}");
        // Sending back via standard output
        Console.WriteLine(number);
    }

    private static void RunTcpMode(int port)
    {
        // Connecting to the local Rust server
        using var client = new TcpClient("127.0.0.1", port);
        using var stream = client.GetStream();

        var buffer = new byte[8];
        var bytesRead = stream.Read(buffer, 0, buffer.Length);

        if (bytesRead != 8) return;
        var number = BitConverter.ToDouble(buffer, 0);
        Console.Error.WriteLine($"[C# Worker] Received via TCP: {number}");
        stream.Write(buffer, 0, buffer.Length);
    }

    private static void RunSharedMemoryMode(string filePath)
    {
        // Opening the memory region created by Rust
        using var mmf =
        MemoryMappedFile.CreateFromFile(filePath, FileMode.Open, null, 8,
        ↪  MemoryMappedFileAccess.ReadWrite);
        using var accessor = mmf.CreateViewAccessor();

        var number = accessor.ReadDouble(0);
        Console.Error.WriteLine($"[C# Worker] Received via SHM: {number}");

        // Negating the number to flag that it was processed
        accessor.Write(0, -number);
    }
}
```